PySpark Interview FAQ Guide

Low, Medium and High Level Questions and Answers

93 questions covering PySpark DataFrames, Spark SQL, joins, window functions, performance tuning, Spark internals, streaming, debugging and coding challenges

Level	Who it is for	Questions
Level 1: LOW	Freshers, juniors, screening rounds	32
Level 2: MEDIUM	2 to 5 years, hands-on engineers	32
Level 3: HIGH	Senior, lead and architect roles	29

Note: Spark defaults and features vary between versions (examples here follow Spark 3.x and 4.x). Code examples use the DataFrame API, with functions imported as F (from pyspark.sql import functions as F). Always confirm exact defaults in the documentation for the version you use.

How to Use This Guide

Read the levels in order. Level 1 builds the vocabulary, Level 2 builds working skill, and Level 3 builds the depth and judgement that senior interviews test. Every answer is written the way you should say it aloud: a clear definition first, then how it works, then an example or code, then the trade-off.

A reliable 5-step formula for answering any question

- **Define** it in one sentence, in plain words.
- **Explain how it works** in two or three sentences, naming the key mechanism.
- **Give an example**: a command, a code snippet, or a real use case from your own work.
- **State the trade-off**: when you would not use it, the cost, or the limit.
- **Connect to experience**: "In my project we used this to ..." (always prepare real examples).

When you do not know the answer

Say what you do know, reason out loud from first principles, and state how you would verify it ("I would check the Spark UI for spill and skew, then look at the table file sizes"). Interviewers value structured thinking more than a perfect memory. Never invent facts; it is better to say "I have not used that feature in production, but I understand it works like this".

Contents

Level 1: LOW (Foundations) 4

Section A: Spark and PySpark Fundamentals 4

Section B: Creating DataFrames and Reading and Writing Data 6

Section C: Everyday DataFrame Operations 9

Section D: Performance Basics and Good Practice 17

Level 2: MEDIUM (Practitioner) 21

Section A: Execution Model, Shuffles and Partitions 21

Section B: Joins and Optimisation 23

Section C: Window Functions, Aggregations and Nested Data 25

Section D: UDFs, Data Sources and Memory 28

Section E: Streaming, Delta and Testing 30
Section F: Coding Questions 32

Level 3: HIGH (Advanced and Architect) 38

Section A: Spark Internals 38
Section B: Performance and File Layout 40
Section C: Advanced Streaming and Pipelines 42
Section D: Debugging and Production Issues 45
Section E: Coding Challenges (Advanced) 48
Section F: Engineering Judgement and Behavioural Questions 52

Appendix A: Rapid-Fire Reference (Defaults and Numbers) 55

Appendix B: Final Interview Checklist 55

Level 1: LOW (Foundations)

Fundamentals every candidate must know cold: Spark architecture, DataFrames, reading and writing data, everyday transformations, joins and basic performance. Typical for freshers, juniors and screening rounds.

Section A: Spark and PySpark Fundamentals

Q1. What is PySpark and why is it used instead of pandas?

Apache Spark is a distributed data-processing engine. **PySpark** is its Python API, so you write Python code that Spark runs in parallel across many machines.

pandas works on one machine and needs the whole dataset in that machine's memory, so it is great up to a few GB. PySpark splits data into **partitions** processed in parallel on a cluster, so it handles hundreds of GB or TB, and it can spill to disk when data is larger than memory. PySpark also supports SQL, streaming, machine learning (MLlib) and many file formats (Parquet, ORC, Avro, JSON, CSV, Delta).

Simple rule: small data fits pandas; big data needs PySpark.

Q2. Explain the Spark architecture: driver, executors and cluster manager.

- **Driver**: the process that runs your main program (SparkSession). It builds the execution plan (DAG), splits it into stages and tasks, schedules them and collects results.
- **Executors**: worker processes on the cluster nodes. They run tasks on data partitions, keep cached data in memory and send results back to the driver.
- **Cluster manager**: allocates resources. Options are Standalone, YARN, Kubernetes and (older) Mesos. In local mode everything runs in one JVM.

Flow: you submit code, the driver asks the cluster manager for executors, the driver sends tasks to executors, and executors process partitions in parallel.

Q3. What are SparkSession and SparkContext? How do you create a SparkSession?

SparkContext is the original entry point that connects to the cluster and creates RDDs. **SparkSession** (Spark 2.0+) is the unified entry point for DataFrames, SQL, streaming and catalog access, and it contains the SparkContext (`spark.sparkContext`).

```
from pyspark.sql import SparkSession

spark = (SparkSession.builder
         .appName("my_app")
         .master("local[*]")       # local mode, all cores
         .getOrCreate())
```

In Databricks and most notebooks, a SparkSession called `spark` already exists, so you do not create it. Use `getOrCreate()` so you reuse an existing session instead of making a second one.

Q4. What is the difference between RDD, DataFrame and Dataset?

- **RDD** (Resilient Distributed Dataset): the low-level API, a distributed collection of Python or JVM objects. It has no schema and Spark cannot optimise it, so it is slower and rarely needed now.
- **DataFrame**: a distributed table with named, typed columns. Spark's Catalyst optimizer and Tungsten engine optimise it, so it is faster and is the standard in PySpark.
- **Dataset**: a typed DataFrame for Scala and Java only; it is not available in Python.

In PySpark, use DataFrames (or Spark SQL) in almost every case.

Q5. What are transformations and actions? What is lazy evaluation?

Transformations create a new DataFrame from an existing one but do not execute anything: `select`, `filter`, `withColumn`, `join`, `groupBy`. **Actions** trigger execution and return a result or write data: `show`, `count`, `collect`, `take`, `write`.

Spark is **lazy**: transformations only build a plan. When an action runs, Spark optimises the entire plan and then executes it. This allows filter pushdown, column pruning and combining steps. DataFrames are also **immutable**: every transformation returns a new DataFrame.

```
df2 = df.filter("age > 30").select("name")   # nothing runs yet
df2.show()                                    # action: runs now
```

Q6. What are narrow and wide transformations?

A **narrow** transformation produces each output partition from a single input partition, so no data moves across the network: `select`, `filter`, `withColumn`, `map`, `union`.

A **wide** transformation needs data from many input partitions, so Spark performs a **shuffle** (writes data to disk and sends it across the network): `groupBy`, `join` (non-broadcast), `distinct`, `orderBy`, `repartition`. Shuffles are the most expensive part of a Spark job, and each shuffle creates a new **stage**.

Q7. What is a partition in Spark? How do you check the number of partitions?

A partition is a chunk of the data that is processed by one task on one core. More partitions mean more parallelism, but too many tiny partitions add scheduling overhead, and too few large ones cause memory pressure. A good partition size is roughly 100 to 200 MB.

```
df.rdd.getNumPartitions()          # number of partitions
df.repartition(8).rdd.getNumPartitions()
```

When reading files, the number of partitions is driven by file sizes (`spark.sql.files.maxPartitionBytes`, default 128 MB). After a shuffle it is `spark.sql.shuffle.partitions` (default 200).

Q8. What are the different ways to run a PySpark program?

- **Notebook** (Databricks, Jupyter): interactive development.
- `pyspark` **shell**: interactive command line with `spark` ready.
- `spark-submit`: submit a script to a cluster for production.
- **Local mode** (`master("local[*]")`): runs on a single machine for testing.

```
spark-submit --master yarn --deploy-mode cluster \
  --num-executors 10 --executor-memory 8g --executor-cores 4 \
  my_job.py
```

Deploy modes: in **client** mode the driver runs on the machine that submits; in **cluster** mode the driver runs inside the cluster, which is preferred for production.

Section B: Creating DataFrames and Reading and Writing Data

Q9. How do you create a DataFrame in PySpark?

```
# 1. From a Python list with column names
data = [(1, "Asha", 50000), (2, "Ravi", 60000)]
df = spark.createDataFrame(data, ["id", "name", "salary"])

# 2. From an explicit schema
from pyspark.sql.types import *
schema = StructType([
    StructField("id", IntegerType(), False),
    StructField("name", StringType(), True),
    StructField("salary", DoubleType(), True)])
df = spark.createDataFrame(data, schema)

# 3. From pandas, or by reading files
df = spark.createDataFrame(pandas_df)
df = spark.read.parquet("/path/to/data")
```

Providing the schema explicitly is faster and safer than letting Spark infer it.

Q10. How do you read and write CSV, JSON and Parquet files?

```
df = (spark.read
      .option("header", True)
      .option("inferSchema", True)
      .csv("/data/sales.csv"))

j = spark.read.json("/data/events.json")
p = spark.read.parquet("/data/sales_parquet")

df.write.mode("overwrite").parquet("/out/sales")
df.write.mode("append").partitionBy("year").parquet("/out/sales")
```

Write modes: `append` (add rows), `overwrite` (replace), `ignore` (do nothing if data exists) and `error` or `errorifexists` (the default; fail if data exists). JSON files with one record per line are the default; use `.option("multiLine", True)` for a single JSON array or pretty-printed JSON.

Q11. Why should you define an explicit schema instead of using inferSchema?

`inferSchema=True` makes Spark read the data an extra time to guess types. That is slow on large data and may guess wrong (for example, a column of IDs with leading zeros becomes an integer, or a date becomes a string). It can also give a different schema on different runs if the data changes.

An explicit `StructType` schema avoids the extra scan, gives stable, predictable types, lets bad data be detected, and documents the expected structure. Parquet and Delta store their schema in the files, so they do not need inference.

Q12. How do you inspect a DataFrame?

```
df.printSchema()        # column names, types, nullability
df.show(5, truncate=False)   # first rows (action)
df.columns              # list of column names
df.dtypes               # list of (name, type)
df.count()              # number of rows (action)
df.describe().show()    # count, mean, stddev, min, max
df.summary().show()     # adds percentiles
```

`printSchema()` and `columns` do not scan the data, while `count()` and `show()` do.

Q13. What is the difference between show(), collect(), take(), head(), first() and toPandas()?

- `show(n)`: prints n rows (default 20) on the driver; nothing is returned.
- `take(n)` / `head(n)`: returns a list of n Row objects.
- `first()`: returns the first Row.
- `collect()`: returns **all** rows to the driver as a list. Dangerous on big data, because it can cause a driver out-of-memory error.
- `toPandas()`: converts the whole DataFrame to a pandas DataFrame on the driver, with the same risk.

Use `limit(n)` before `toPandas()` or `collect()`, and prefer writing results to storage when they are large.

Section C: Everyday DataFrame Operations

Q14. What are the most common DataFrame transformations?

```
from pyspark.sql import functions as F

df.select("id", "name")                          # choose columns
df.filter(F.col("salary") > 50000)               # or df.where(...)
df.withColumn("bonus", F.col("salary") * 0.1)  # add or replace column
df.withColumnRenamed("name", "emp_name")
df.drop("temp_col")
df.distinct()                                    # unique rows
df.dropDuplicates(["id"])                        # unique by columns
df.orderBy(F.col("salary").desc())
df.limit(10)
```

Each call returns a new DataFrame, so you chain them.

Q15. How do you filter with multiple conditions in PySpark?

Use `&` (and), `|` (or) and `~` (not), and put **each condition in parentheses**, because of Python operator precedence. Do not use Python's `and`, `or` or `not`.

```
df.filter((F.col("age") > 25) & (F.col("city") == "Hyderabad"))
df.filter((F.col("dept") == "IT") | (F.col("dept") == "HR"))
df.filter(F.col("dept").isin("IT", "HR"))
df.filter(F.col("name").like("A%"))
df.filter(F.col("salary").between(40000, 80000))
df.filter(~F.col("city").isin("Pune", "Delhi"))
df.filter("age > 25 AND city = 'Hyderabad'")   # SQL string also works
```

Q16. How do you use when/otherwise, cast, lit and alias?

```
df2 = (df
  .withColumn("grade",
      F.when(F.col("score") >= 90, "A")
       .when(F.col("score") >= 75, "B")
       .otherwise("C"))
  .withColumn("score_int", F.col("score").cast("int"))
  .withColumn("country", F.lit("India")))

df.select(F.col("salary").alias("pay"))
```

- `when/otherwise` is the DataFrame version of CASE WHEN.
- `cast` changes a column's data type.
- `lit` creates a constant column.
- `alias` renames a column in a `select` or `agg`.

Q17. How do you handle null values?

```
df.filter(F.col("email").isNull())          # find nulls
df.filter(F.col("email").isNotNull())
df.dropna()                                 # drop rows with any null
df.dropna(subset=["id", "email"])           # only these columns
df.fillna({"salary": 0, "city": "Unknown"}) # replace nulls
df.withColumn("x", F.coalesce("a", "b", F.lit(0)))  # first non-null
```

Remember: any comparison with null gives null (not true or false), so `col == None` does not work; use `isNull()`. Aggregations such as `sum` and `avg` ignore nulls, and `count(col)` counts only non-null values while `count("*")` counts all rows.

Q18. How do you do groupBy and aggregations?

```
result = (df.groupBy("dept")
   .agg(F.count("*").alias("employees"),
        F.sum("salary").alias("total_salary"),
        F.avg("salary").alias("avg_salary"),
        F.max("salary").alias("max_salary"),
        F.countDistinct("city").alias("cities")))

result.filter(F.col("employees") > 5)      # like HAVING
df.agg(F.sum("salary")).show()             # whole-table aggregate
```

`groupBy` is a wide transformation (it shuffles). Filtering before grouping reduces the amount of data shuffled.

Q19. What is the difference between orderBy and sort, and how do you sort descending?

`orderBy` and `sort` are aliases and do the same global sort. Sorting is a wide operation (it shuffles and range-partitions data), so avoid it unless needed.

```
df.orderBy("salary")                                    # ascending
df.orderBy(F.col("salary").desc())                      # descending
df.orderBy(F.col("dept").asc(), F.col("salary").desc())
df.orderBy(F.col("x").desc_nulls_last())
```

`sortWithinPartitions` sorts only inside each partition, which is much cheaper when a global order is not required.

Q20. What are the types of joins and how do you write them?

```
emp.join(dept, "dept_id", "inner")                    # same column name
emp.join(dept, emp.dept_id == dept.id, "left")        # different names
```

Join types: **inner** (matches only), **left**, **right**, **full** (outer), **left_semi** (left rows that have a match, only left columns), **left_anti** (left rows with no match) and **cross**.

When you join on a condition and both tables have the same column name, the result has duplicate column names, which causes "ambiguous reference" errors later. Fix it by joining on the column name (`"dept_id"`), aliasing the DataFrames (`emp.alias("e")`), or dropping one of the columns.

Q21. What is the difference between union and unionByName?

`union` (same as the older `unionAll`) combines two DataFrames **by column position** and does not remove duplicates, unlike SQL's UNION. If the column order differs you silently get wrong data. `unionByName` matches columns **by name**, which is safer, and `allowMissingColumns=True` fills missing columns with null.

```
a.unionByName(b, allowMissingColumns=True)
a.union(b).distinct()      # SQL-style UNION (removes duplicates)
```

Q22. Which string and date functions should you know?

```
F.upper("name"), F.lower("name"), F.trim("name"), F.length("name")
F.concat("a", F.lit("-"), "b"), F.concat_ws("-", "a", "b")
F.substring("name", 1, 3), F.regexp_replace("phone", "[^0-9]", "")
F.split("tags", ","), F.initcap("name")

F.to_date("d", "yyyy-MM-dd"), F.to_timestamp("t")
F.date_format("d", "yyyy-MM"), F.current_date(), F.current_timestamp()
F.datediff("end", "start"), F.add_months("d", 1), F.date_add("d", 7)
F.year("d"), F.month("d"), F.dayofweek("d")
```

Date patterns use Java-style letters: `yyyy`, `MM` (month), `dd`, `HH`, `mm` (minutes), `ss`. A frequent mistake is writing `mm` for month.

Q23. How do you use SQL with DataFrames?

Register a DataFrame as a temporary view and query it with `spark.sql`.

```
df.createOrReplaceTempView("employees")

top = spark.sql('''
    SELECT dept, AVG(salary) AS avg_sal
    FROM employees
    GROUP BY dept
    HAVING AVG(salary) > 50000
''')
```

The SQL API and the DataFrame API compile to the same optimised plan, so performance is identical. A temp view lives only for the current Spark session; `createGlobalTempView` makes it visible across sessions in the same application (accessed as `global_temp.name`).

Q24. What do count(*), count(col) and countDistinct(col) return?

- `count("*")` or `count(F.lit(1))`: the total number of rows, including nulls.
- `count("col")`: the number of rows where `col` is **not null**.
- `countDistinct("col")`: the number of distinct non-null values; for huge data, `approx_count_distinct` is much cheaper and about 5 percent accurate by default.

Q25. How do you work with arrays and explode?

`explode` turns each element of an array (or map) into its own row.

```
df = spark.createDataFrame([(1, ["a", "b"]), (2, ["c"])], ["id", "tags"])
df.select("id", F.explode("tags").alias("tag")).show()
# id=1 tag=a ; id=1 tag=b ; id=2 tag=c
```

Useful companions: `explode_outer` (keeps rows with null or empty arrays), `size("tags")`, `array_contains("tags", "a")`, `collect_list` and `collect_set` (the reverse: group values back into an array).

Q26. What are the nested data types (StructType, ArrayType, MapType) and how do you access them?

- **StructType**: a record with named fields. Access with dot notation: `F.col("address.city")`.
- **ArrayType**: a list of same-typed values. Access by index: `F.col("tags")[0]` or `getItem(0)`.
- **MapType**: key to value pairs. Access `F.col("props")["color"]` or `getItem("color")`.

```
df.select("address.city", F.col("tags")[0], F.col("props")["color"])
df.select("address.*")          # expand all struct fields to columns
```

Section D: Performance Basics and Good Practice

Q27. What is the difference between repartition() and coalesce()?

- `repartition(n)`: performs a **full shuffle**; can increase or decrease partitions and produces evenly sized partitions. `repartition(n, "col")` partitions by column values.
- `coalesce(n)`: only **reduces** partitions by merging neighbours, with no full shuffle. It is cheaper but may leave uneven partitions.

Use `coalesce` to cut the number of output files; use `repartition` when you need more parallelism or an even distribution before heavy work.

Q28. What are cache() and persist()? When should you use them?

They keep a DataFrame in memory (and optionally disk) after it is first computed, so later actions do not recompute it from the source.

```
from pyspark import StorageLevel
df.cache()                      # MEMORY_AND_DISK for DataFrames
df.persist(StorageLevel.DISK_ONLY)
df.count()                                  # first action fills the cache
df.unpersist()                              # release the memory
```

Use caching when the **same DataFrame is used by several actions** (for example, iterative loops or one dataset feeding several outputs). Do not cache everything: it uses memory that other work needs, and it only helps when data is reused.

Q29. What are Parquet, ORC, Avro, CSV and JSON? Which one should you choose?

- **Parquet**: columnar, compressed, stores the schema, supports column pruning and predicate pushdown. The default and best choice for analytics.
- **ORC**: columnar like Parquet, popular in the Hive ecosystem.
- **Avro**: row-based, good for streaming and write-heavy pipelines (Kafka) and schema evolution.
- **CSV** and **JSON**: text formats, human-readable but large, slow and without strong schemas; good for exchange, not for storage of big data.

For a Lakehouse, store tables in **Delta** (which uses Parquet files plus a transaction log).

Q30. What is partitionBy when writing data? What are its risks?

`df.write.partitionBy("year", "month")` creates a folder per value (`year=2026/month=10/`). Queries that filter on those columns read only the matching folders (**partition pruning**).

Choose low-cardinality columns such as date, country or year. Partitioning by a high-cardinality column (such as `user_id`) creates millions of tiny files and slows everything down. Also, partition columns are stored in folder names, not in the data files.

Q31. What are some beginner best practices in PySpark?

- Prefer built-in functions (`pyspark.sql.functions`) over Python UDFs.
- Avoid `collect()` and `toPandas()` on large data.
- Filter and select columns as early as possible.
- Use explicit schemas for CSV and JSON.
- Use Parquet or Delta, not CSV, for stored data.
- Avoid unnecessary shuffles and `distinct`/`orderBy` on big data.
- Do not use loops over rows; think in whole-column operations.
- Cache only reused DataFrames, and unpersist when done.
- Name columns clearly and keep transformations in small, testable functions.

Q32. What are UDFs and why should you avoid them when possible?

A UDF (user-defined function) lets you apply custom Python logic to a column.

```
from pyspark.sql.functions import udf
from pyspark.sql.types import StringType

@udf(returnType=StringType())
def clean(s):
    return s.strip().title() if s else None

df.withColumn("name_clean", clean("name"))
```

A Python UDF is slow because every row is serialised from the JVM to a Python process and back, and Catalyst cannot optimise inside it. Use built-in functions first, then **pandas UDFs** (vectorised), and a plain UDF only as a last resort.

Level 2: MEDIUM (Practitioner)

Hands-on working knowledge for 2 to 5 years of experience: execution model, shuffles, joins and optimisation, window functions, nested data, UDFs, memory, streaming, testing and coding questions.

Section A: Execution Model, Shuffles and Partitions

Q1. Explain how Spark executes a job: DAG, jobs, stages and tasks.

When you call an **action**, Spark creates a **job**. The driver turns your transformations into a logical plan, optimises it with Catalyst, makes a physical plan, and builds a **DAG** (directed acyclic graph) of operations.

The DAG is split into **stages** at **shuffle boundaries** (wide transformations). Each stage contains one **task per partition**, and the tasks of a stage run in parallel on the executor cores. A stage must finish before the next one that depends on its shuffle output can start.

Example: `read -> filter -> groupBy -> count` has two stages: stage 1 (read, filter and partial aggregation) and stage 2 (final aggregation after the shuffle).

Q2. What is a shuffle, why is it expensive, and how do you reduce it?

A shuffle redistributes data across partitions so that rows with the same key end up together. Each map task writes its output to local disk, and reduce tasks fetch the pieces over the network. It costs disk I/O, network I/O, serialisation and sorting, and it is where most Spark slowness and out-of-memory errors come from.

Operations that shuffle: `groupBy`, `join` (except broadcast), `distinct`, `orderBy`, `repartition`, window functions with partitioning.

How to reduce it:

- Filter and select columns **before** the shuffle.
- Use **broadcast joins** for small tables.
- Use `reduceByKey` or DataFrame aggregations (which pre-aggregate on the map side) instead of `groupByKey`.
- Avoid unnecessary `distinct`, `orderBy` and repartitions.
- Pre-partition or bucket data that is joined repeatedly on the same key.
- Tune `spark.sql.shuffle.partitions` and let AQE coalesce.

Q3. How do you choose the right number of partitions? What is spark.sql.shuffle.partitions?

`spark.sql.shuffle.partitions` (default 200) is the number of partitions created after a shuffle in DataFrame and SQL operations. The default is too small for terabytes (huge partitions that spill) and too big for small data (200 tiny tasks).

Rules of thumb: aim for about **100 to 200 MB per partition**, and at least **2 to 4 times the total number of cores** so that all cores stay busy. With **Adaptive Query Execution** (on by default since Spark 3.2), you can set a higher value and Spark will coalesce small shuffle partitions at runtime.

```
spark.conf.set("spark.sql.shuffle.partitions", 400)
spark.conf.set("spark.sql.adaptive.enabled", True)
```

Q4. repartition vs coalesce vs repartitionByRange: when do you use each?

- `repartition(n)`: full shuffle, round-robin, even-sized partitions. Use to increase parallelism or fix imbalance.
- `repartition(n, "key")`: hash-partitions by key so the same key is in the same partition; useful before a join, groupBy or a partitioned write to control the file count.
- `repartitionByRange(n, "col")`: range-partitions so that partitions hold sorted ranges of values; useful before writing sorted output.
- `coalesce(n)`: only reduces partitions with no full shuffle; use to reduce the number of output files at the end.

Writing one file per partition value:
`df.repartition("date").write.partitionBy("date")...` avoids many small files in each folder.

Q5. How do you read the Spark UI to find a bottleneck?

- **Jobs and Stages tabs**: find the stage that takes the longest. Open it and compare the **task duration distribution**: if the max is far above the median, you have **skew**.
- **Shuffle Read/Write size**: huge shuffles point to expensive joins or aggregations.
- **Spill (memory and disk)**: shows partitions too large for memory.
- **SQL tab**: the plan with metrics per operator (rows output, scan time), where you can see which join strategy was used.
- **Executors tab**: GC time (high GC means memory pressure), failed tasks and uneven task counts.
- **Storage tab**: what is cached and how much memory it uses.

Say it like this in interviews: "I find the slowest stage, check skew, spill and shuffle size, and then decide the fix."

Q6. What is data skew and how do you fix it?

Skew means a few keys hold much more data than others, so a few tasks run far longer while other cores sit idle. Symptoms: one task running for a long time, spill, or out-of-memory on one executor.

Fixes:

- **AQE skew join** (`spark.sql.adaptive.skewJoin.enabled`): splits skewed partitions automatically.
- **Broadcast** the smaller table so there is no shuffle.
- **Filter early** or remove junk keys such as null or "unknown" that carry most of the rows.
- **Salting**: add a random suffix to the hot key to spread it, then aggregate in two steps.

```
salted = df.withColumn("salt", (F.rand() * 10).cast("int"))
step1 = salted.groupBy("key", "salt").agg(F.sum("v").alias("s"))
final = step1.groupBy("key").agg(F.sum("s").alias("total"))
```

Section B: Joins and Optimisation

Q7. What is a broadcast join and when do you use it?

In a broadcast join, Spark sends a copy of the small table to every executor, so each executor joins its partitions of the big table locally and the **big table is not shuffled**. It is the fastest join type for a big table joined with a small one.

Spark broadcasts automatically when a table is smaller than `spark.sql.autoBroadcastJoinThreshold` (default 10 MB). You can force it with a hint:

```
from pyspark.sql.functions import broadcast
result = orders.join(broadcast(countries), "country_id")
```

Do not broadcast something that is too large: it can run the driver or executors out of memory or hit the `spark.sql.broadcastTimeout` (default 300 seconds).

Q8. How does Spark choose a join strategy?

- **Broadcast Hash Join**: one side is small; no shuffle of the large side. Chosen when size is under the threshold or when hinted.
- **Sort-Merge Join**: the default for two large tables on an equi-join. Both sides are shuffled and sorted by key, then merged; it can spill to disk, so it scales well.
- **Shuffle Hash Join**: shuffles both sides and builds a hash table on the smaller side per partition.
- **Broadcast Nested Loop / Cartesian**: for non-equi or cross joins; very expensive.

You can influence the choice with hints: `df.hint("broadcast")`, `merge`, `shuffle_hash`. Run `df.explain()` to see which join was used.

Q9. What does explain() show and how do you read it?

`df.explain()` prints the physical plan; `df.explain("formatted")` or `df.explain(mode="extended")` shows the parsed, analysed, optimised and physical plans.

```
df.filter("year = 2026").groupBy("dept").count().explain("formatted")
```

Read the plan **from the bottom up**. Look for: `FileScan` with `PushedFilters` and `PartitionFilters` (confirms pruning and pushdown), `Exchange` (a shuffle), `BroadcastHashJoin` or `SortMergeJoin`, `HashAggregate` (partial and final) and `AdaptiveSparkPlan`. Your goal is to see filters pushed down and as few Exchange nodes as possible.

Q10. What are the Catalyst optimizer and Tungsten?

Catalyst is Spark SQL's query optimizer. It resolves columns and types (analysis), applies rule-based optimisations such as **predicate pushdown**, **column pruning**, constant folding and join reordering, chooses physical operators (cost-based using statistics) and generates code.

Tungsten is the execution engine improvement: binary off-heap memory format, cache-friendly layout and **whole-stage code generation**, which fuses operators into one tight loop.

Both work only on DataFrame, Dataset and SQL operations. This is why the DataFrame API is faster than RDDs, and why Python UDFs (opaque to Catalyst) are slower.

Q11. What are predicate pushdown, column pruning and partition pruning?

- **Column pruning**: for columnar formats such as Parquet, Spark reads only the columns the query needs.
- **Predicate pushdown**: filters are pushed into the file scan, so Parquet row groups whose min and max statistics cannot match are skipped.
- **Partition pruning**: if data is partitioned by a column (such as `year`) and you filter on it, only the matching folders are read.

All three reduce the amount of data read. They work best with Parquet or Delta and with simple filters on raw columns; wrapping a column in a function in the filter (for example `year(date_col) = 2026` on a non-partition column) can prevent pushdown.

Q12. What are broadcast variables and accumulators?

Broadcast variable: a read-only value sent once to each executor and cached, instead of shipping it with every task. Use it for lookup dictionaries in RDD or UDF code.

Accumulator: a shared variable that tasks can only add to, and only the driver can read; used for counters such as bad-record counts.

```
lookup = spark.sparkContext.broadcast({"IN": "India", "US": "USA"})
bad = spark.sparkContext.accumulator(0)

def f(code):
    return lookup.value.get(code, "NA")
```

Caution: accumulator updates inside transformations may be applied more than once if a task is retried, so only rely on them inside actions.

Section C: Window Functions, Aggregations and Nested Data

Q13. Explain window functions with an example (top N per group).

A window function computes a value over a group of related rows without collapsing them. You define a window with `partitionBy` (groups), `orderBy` (order within the group) and optionally a frame.

```
from pyspark.sql import Window

w = Window.partitionBy("dept").orderBy(F.col("salary").desc())

top2 = (emp
   .withColumn("rank", F.dense_rank().over(w))
   .filter("rank <= 2"))
```

Ranking functions: `row_number` (unique 1,2,3), `rank` (ties share a rank and leave gaps: 1,1,3), `dense_rank` (ties share a rank with no gaps: 1,1,2). Others: `lag`, `lead`, `sum`, `avg`, `first`, `last`.

Q14. How do you compute a running total, lag/lead and moving average?

```
w = Window.partitionBy("cust").orderBy("order_date")

df = (df
  .withColumn("running_total", F.sum("amount").over(w))
  .withColumn("prev_amount", F.lag("amount", 1).over(w))
  .withColumn("next_amount", F.lead("amount", 1).over(w)))

w3 = w.rowsBetween(-2, 0)     # current and previous 2 rows
df = df.withColumn("moving_avg_3", F.avg("amount").over(w3))
```

Important: with an `orderBy` the default frame is `RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW` (so ties are included together), and without `orderBy` the frame is the whole partition.

Q15. How do pivot, rollup and cube work?

```
sales.groupBy("year").pivot("quarter", ["Q1", "Q2", "Q3", "Q4"]) \
     .agg(F.sum("amount"))              # quarters become columns

sales.rollup("region", "city").agg(F.sum("amount"))  # subtotals + total
sales.cube("region", "city").agg(F.sum("amount"))    # all combinations
```

Passing the list of pivot values avoids an extra job that finds the distinct values, so it is faster. `rollup` gives hierarchical subtotals, while `cube` gives subtotals for every combination of the columns. Use `grouping()` or `grouping_id()` to tell subtotal rows from real nulls.

Q16. collect_list vs collect_set, and how do you group values into an array?

`collect_list` gathers all values of a group into an array (keeps duplicates, order not guaranteed). `collect_set` gathers only the distinct values.

```
df.groupBy("cust").agg(
    F.collect_list("product").alias("all_products"),
    F.collect_set("product").alias("unique_products"))
```

Be careful with very large groups, because the whole array for one key must fit in one executor's memory. For ordered arrays, sort inside using `sort_array` or collect structs and sort them.

Q17. How do you handle JSON and nested data (from_json, explode, flattening)?

```
from pyspark.sql.types import *
schema = StructType([
    StructField("id", IntegerType()),
    StructField("items", ArrayType(StructType([
        StructField("sku", StringType()),
        StructField("qty", IntegerType())])))])

parsed = df.withColumn("j", F.from_json("payload", schema))
flat = (parsed
   .select("j.id", F.explode("j.items").alias("item"))
   .select("id", "item.sku", "item.qty"))
```

`from_json` parses a JSON string column using a schema; `to_json` does the reverse; `explode` flattens arrays into rows; `col("struct.field")` and `struct.*` flatten structs. Use `explode_outer` if rows with empty arrays must be kept.

Q18. What is the difference between dropDuplicates and using row_number to deduplicate?

`dropDuplicates(["id"])` keeps **one arbitrary row** per key, so which row survives is not guaranteed. When you need a rule, such as the latest record, use a window:

```
w = Window.partitionBy("id").orderBy(F.col("updated_at").desc())
latest = (df.withColumn("rn", F.row_number().over(w))
            .filter("rn = 1").drop("rn"))
```

`distinct()` removes only fully identical rows. For exact duplicates over all columns, `distinct()` or `dropDuplicates()` with no arguments is enough.

Section D: UDFs, Data Sources and Memory

Q19. UDF vs pandas UDF vs built-in functions: what is the difference?

- **Built-in functions**: run inside the JVM, fully optimised by Catalyst and codegen. Always the first choice.
- **Python UDF**: rows are pickled and sent one by one to a Python worker, then back. Slow, and opaque to the optimizer.
- **pandas UDF** (vectorised UDF): data is sent in **Apache Arrow** batches and processed as pandas Series, which is typically many times faster than a plain UDF.

```
import pandas as pd
from pyspark.sql.functions import pandas_udf

@pandas_udf("double")
def with_tax(price: pd.Series) -> pd.Series:
    return price * 1.18

df.withColumn("gross", with_tax("price"))
```

Related: `applyInPandas` (grouped map) and `mapInPandas` for DataFrame-to-DataFrame logic.

Q20. How do you handle corrupt or malformed records when reading files?

The reader **mode** option controls behaviour:

- `PERMISSIVE` (default): keeps the row, sets bad fields to null and stores the raw text in a corrupt-record column (`_corrupt_record`) if you include it in the schema.
- `DROPMALFORMED`: silently drops bad rows.
- `FAILFAST`: throws an exception on the first bad row.

```
df = (spark.read.schema(schema_with_corrupt_col)
      .option("mode", "PERMISSIVE")
      .option("columnNameOfCorruptRecord", "_corrupt_record")
      .json("/data/in"))
```

Best practice: use PERMISSIVE, route rows with a non-null corrupt column to a quarantine location, and alert on the count.

Q21. Parquet vs ORC vs Avro vs Delta: how do you choose?

- **Parquet / ORC**: columnar, best for analytical reads (column pruning, predicate pushdown, good compression).
- **Avro**: row-based; good for write-heavy, streaming and Kafka pipelines, with strong schema evolution.
- **Delta Lake**: Parquet plus a transaction log: ACID transactions, updates and deletes, MERGE, time travel and schema enforcement. It is the best default for tables in a Lakehouse.
- **CSV / JSON**: interchange only.

Parquet files are immutable, so you cannot update a row in place; Delta gives you that capability on top of Parquet.

Q22. How is memory used in a Spark executor, and how do you configure resources?

Executor memory is split into: **reserved** (about 300 MB), **unified memory** (`spark.memory.fraction`, default 0.6 of the remaining heap) shared by **execution** (shuffles, joins, sorts, aggregations) and **storage** (cached data), and **user memory** for your own objects. Outside the heap there is **memory overhead**, which matters a lot for PySpark because the Python workers live there.

```
spark-submit \
  --executor-memory 8g --executor-cores 4 --num-executors 10 \
  --conf spark.executor.memoryOverhead=2g \
  --conf spark.driver.memory=4g my_job.py
```

Also know **dynamic allocation** (`spark.dynamicAllocation.enabled`), which adds and removes executors with load.

Q23. What is the difference between cache and checkpoint?

`cache()` keeps data in memory or disk but **keeps the lineage**: if an executor is lost, Spark can recompute the data from the beginning. `checkpoint()` writes the data to reliable storage and **truncates the lineage**.

```
spark.sparkContext.setCheckpointDir("/chk")
df = df.checkpoint()          # eager by default; cuts the lineage
```

Use checkpoint in very long or iterative pipelines (for example loops that build up hundreds of transformations), where the plan itself becomes huge and slow to optimise.

Q24. What are RDD operations such as map, flatMap, reduceByKey and groupByKey? Why is reduceByKey better?

- `map` returns exactly one output element per input; `flatMap` can return zero or more (used to split lines into words).
- `reduceByKey(f)` combines values for each key **on each partition first** (map-side combine) and then shuffles only the partial results.
- `groupByKey()` shuffles **all** values for each key and only then groups them, so it moves much more data and can cause out-of-memory errors.

```
words = sc.textFile("in.txt").flatMap(lambda l: l.split())
counts = words.map(lambda w: (w, 1)).reduceByKey(lambda a, b: a + b)
```

In DataFrames, `groupBy().agg()` already does the map-side partial aggregation automatically.

Section E: Streaming, Delta and Testing

Q25. What is Structured Streaming? Explain triggers, checkpoints, output modes and watermarks.

Structured Streaming treats a live data stream as an unbounded table and processes it with the same DataFrame API in micro-batches.

```
q = (spark.readStream.format("json").schema(schema).load("/in")
     .withWatermark("event_time", "10 minutes")
     .groupBy(F.window("event_time", "5 minutes")).count()
     .writeStream.outputMode("update")
     .option("checkpointLocation", "/chk/q1")
     .trigger(processingTime="1 minute")
     .format("delta").start("/out"))
```

- **Trigger**: when batches run (`processingTime`, or `availableNow=True` to process all available data and stop).
- **Checkpoint**: stores offsets and state so the query can resume after failure.
- **Output modes**: `append`, `update`, `complete`.
- **Watermark**: how late data may arrive; it lets Spark discard old state.

Q26. How do you use Delta Lake from PySpark (read, write, merge)?

```
df.write.format("delta").mode("overwrite").saveAsTable("sales.orders")
spark.read.table("sales.orders")

from delta.tables import DeltaTable
t = DeltaTable.forName(spark, "sales.orders")
(t.alias("t").merge(updates.alias("s"), "t.id = s.id")
  .whenMatchedUpdateAll()
  .whenNotMatchedInsertAll()
  .execute())

spark.sql("DESCRIBE HISTORY sales.orders")
spark.read.option("versionAsOf", 3).table("sales.orders")   # time travel
```

Delta adds ACID transactions, upserts and time travel on top of Parquet files.

Q27. How do you test PySpark code?

- Keep logic in **small functions that take and return DataFrames**, not in one big script.
- Create a small local SparkSession in a pytest fixture, and use tiny hand-made DataFrames.
- Compare actual and expected results (collect to lists, or use a helper library such as chispa).
- Add data-quality tests on real data: null counts, uniqueness of keys, row-count reconciliation.

```
import pytest
from pyspark.sql import SparkSession

@pytest.fixture(scope="session")
def spark():
    return SparkSession.builder.master("local[2]").getOrCreate()

def test_dedup(spark):
    df = spark.createDataFrame([(1, 5), (1, 9)], ["id", "v"])
    assert dedup_latest(df).collect()[0]["v"] == 9
```

Section F: Coding Questions

Q28. Coding: word count with DataFrames.

```
lines = spark.read.text("/data/book.txt")          # column: value

counts = (lines
   .select(F.explode(F.split(F.lower("value"), r"\W+")).alias("word"))
   .filter("word != ''")
   .groupBy("word").count()
   .orderBy(F.desc("count")))
counts.show(10)
```

Steps: split each line into words, explode to one word per row, clean empties, group and count.

Q29. Coding: find the second highest salary in each department.

```
w = Window.partitionBy("dept").orderBy(F.col("salary").desc())

second = (emp
   .withColumn("r", F.dense_rank().over(w))
   .filter("r = 2")
   .select("dept", "name", "salary"))
```

Use `dense_rank` so equal salaries share a rank (the second distinct salary), and `row_number` if you want exactly one row. A department with only one salary returns no row.

Q30. Coding: find customers who have never placed an order.

The cleanest way is a **left anti join**.

```
no_orders = customers.join(orders, "customer_id", "left_anti")
```

Equivalent with a left join: `customers.join(orders, "customer_id", "left").filter(F.col("order_id").isNull())`. The anti join is clearer and usually faster, and returns only the left table's columns.

Q31. Coding: compute a percentage of total per group and per-row difference from the group average.

```
w = Window.partitionBy("dept")

df = (df
  .withColumn("dept_total", F.sum("salary").over(w))
  .withColumn("pct_of_dept",
      F.round(F.col("salary") / F.col("dept_total") * 100, 2))
  .withColumn("diff_from_avg", F.col("salary") - F.avg("salary").over(w)))
```

A window with only `partitionBy` (no `orderBy`) aggregates over the whole group while keeping every row, which avoids a join back to a grouped DataFrame.

Q32. Coding: basic data-quality profile (null counts, duplicates).

```
null_counts = df.select([
    F.sum(F.col(c).isNull().cast("int")).alias(c) for c in df.columns])
null_counts.show()

dups = df.groupBy("id").count().filter("count > 1")
print("duplicate keys:", dups.count())

print("rows:", df.count(), "distinct rows:", df.distinct().count())
```

Running such checks after each pipeline stage, and failing or alerting when thresholds are broken, is a simple and effective data-quality gate.

Level 3: HIGH (Advanced and Architect)

Internals, tuning and scenario questions for senior and lead roles: Catalyst and AQE, memory model, shuffle internals, executor sizing, file layout, stateful streaming, debugging, system design and advanced coding challenges.

Section A: Spark Internals

Q1. Explain the Catalyst optimizer phases, Tungsten and Adaptive Query Execution in depth.

Catalyst has four phases: (1) **analysis**: resolve table, column and type references against the catalog; (2) **logical optimisation**: rule-based rewrites such as predicate pushdown, column pruning, constant folding, filter reordering and join reordering; (3) **physical planning**: generate candidate physical plans and choose by cost (sizes and statistics decide the join strategy); (4) **code generation**: whole-stage codegen compiles a pipeline of operators into one Java method.

Tungsten supplies the efficient binary memory format, off-heap management and cache-aware computation.

AQE (default on since Spark 3.2) re-optimises at shuffle boundaries using real runtime statistics: it **coalesces** small shuffle partitions, **converts** a sort-merge join to a broadcast join when one side turns out to be small, and **splits skewed partitions** in joins (by default a partition is skewed if it is more than 5 times the median and larger than 256 MB). **Dynamic partition pruning** additionally prunes fact-table partitions using the result of a filtered dimension table.

Q2. How does the Spark memory model work, and how do you diagnose OutOfMemory errors?

For each executor JVM: heap minus a **reserved 300 MB**, then `spark.memory.fraction` (0.6) forms **unified memory**, shared between **execution** (shuffle, join, sort and aggregation buffers) and **storage** (cache). `spark.memory.storageFraction` (0.5) is the portion of unified memory protected from eviction by execution. The rest is **user memory**. Additionally there is **off-heap overhead** (`spark.executor.memoryOverhead`, default about 10 percent or at least 384 MB), where Python workers, native libraries and network buffers live; PySpark often needs more.

Diagnosis:

- **Driver OOM**: `collect()`, `toPandas()`, or a huge broadcast. Keep data distributed.

- **Executor OOM**: skewed or oversized partitions, big aggregations or exploding joins. Check spill and max versus median task size.
- **"Container killed by YARN/Kubernetes for exceeding memory"** (not a Java heap error): raise `memoryOverhead` or `spark.executor.pyspark.memory`, and reduce per-task Python memory (smaller Arrow batches).

Fix order: remove data early, fix skew, increase shuffle partitions (smaller partitions), then increase memory, and only last add nodes.

Q3. Explain how shuffle works internally.

Spark uses a **sort-based shuffle**. In the **map side** of a stage, each task writes its output records partitioned by the target reduce partition into a single data file plus an index file on local disk (with map-side combine for aggregations, so `reduceByKey` and DataFrame aggregations send partial results). In the **reduce side** of the next stage, each task fetches its blocks from all map outputs over the network, merges (and for sort-merge joins, sorts) them and processes the result.

Consequences: shuffle files live on executor local disk until the stage output is no longer needed; losing an executor loses its shuffle files unless an **external shuffle service** or remote shuffle storage is used (then the stage may need to be recomputed); the number of shuffle blocks is mappers times reducers, so too many tiny partitions create heavy overhead; and big shuffles cause disk spill and network pressure. This is why reducing shuffles is the top tuning lever.

Q4. How do you size executors for a given cluster? Give a worked example.

Common guidance: about **4 to 5 cores per executor** (many cores per JVM cause HDFS and I/O contention and GC trouble), leave **1 core and about 1 GB per node** for the OS and daemons, and leave resources for the driver or application master. Then derive memory from what is left, subtracting overhead.

Example: 10 nodes, each with 16 cores and 64 GB.

- Usable per node: 15 cores, 63 GB.
- 5 cores per executor gives 3 executors per node, so 30 executors, minus 1 for the application master gives about 29.
- Memory per executor: 63 GB / 3 = 21 GB, minus about 7 to 10 percent overhead gives about `--executor-memory 19g`.
- Parallelism: 29 executors times 5 cores = 145 cores, so use at least 300 to 600 shuffle partitions.

On Kubernetes and Databricks the logic is the same, but node and executor mapping is managed for you. Always validate with the Spark UI (spill, GC time, idle cores) rather than trusting the formula blindly.

Q5. How does Spark handle failures and stragglers? What are lineage, speculation and retries?

Fault tolerance comes from **lineage**: every dataset knows the transformations that produced it, so lost partitions are recomputed. A failed **task** is retried (`spark.task.maxFailures`, default 4); a lost shuffle output triggers recomputation of the map stage; losing the driver kills the application unless it is restarted by the cluster manager (cluster deploy mode with supervise, or the job scheduler).

Stragglers are tasks that run much slower than others. **Speculative execution** (`spark.speculation`, off by default) launches duplicate copies of slow tasks and uses whichever finishes first. It helps with bad nodes but not with data skew (the duplicate is just as slow) and it needs idempotent writes. **Data locality** (`spark.locality.wait`, default 3 seconds) controls how long the scheduler waits to run a task near its data.

For long lineages, use **checkpoint** to cut recomputation and plan size.

Q6. How do PySpark and the JVM interact? Explain Python worker processes, Py4J, Arrow and pandas UDF types.

Your PySpark driver code talks to the JVM through **Py4J**: DataFrame operations are just commands building a JVM plan, so built-in functions run entirely in the JVM at full speed. Python code only executes for **UDFs, RDD lambdas** and driver logic: executors launch **Python worker processes** and stream data between the JVM and Python.

- **Python UDF**: row-by-row pickling, high serialisation cost, no optimisation.
- **Pandas (vectorised) UDF**: data moves as **Arrow** columnar batches (`spark.sql.execution.arrow.maxRecordsPerBatch`, default 10000), and you work on pandas Series, so it is far faster. Variants: Series to Series, `groupBy().applyInPandas()` (grouped map: whole group as a pandas DataFrame), `mapInPandas` (iterator of DataFrames) and `cogroup().applyInPandas`.
- Arrow also speeds `toPandas()` and `createDataFrame(pandas_df)` when enabled (`spark.sql.execution.arrow.pyspark.enabled`).

Memory implication: Python workers use memory outside the JVM heap, so budget `memoryOverhead` for them, and a grouped-map group must fit in one worker's memory.

Section B: Performance and File Layout

Q7. A PySpark job slowed down from 20 minutes to 3 hours. How do you troubleshoot?

- **What changed?** Input volume, new keys, code change, Spark or library version, cluster size, or competing workloads.
- **Find the slow stage** in the Spark UI and classify the symptom:
- One task far longer than others means **skew**.
- Heavy spill means partitions are too large (increase partitions or fix skew).
- Thousands of tiny tasks means **small files** or too many partitions.
- Long scan times mean no pushdown or pruning, or many small files.
- Large shuffle read/write means a bad join strategy or unnecessary shuffles.
- Long gaps with no tasks mean driver-side work such as `collect`, Python loops or a huge plan.
- **Apply the targeted fix**: broadcast, salting, AQE, repartition, compaction, filter earlier, caching reused data, replacing UDFs.
- **Re-measure**, then add monitoring so a regression is detected earlier.

Q8. What causes the small-files problem and how do you fix it?

Sources: too many shuffle partitions writing a small dataset, over-partitioned folders (`partitionBy` on a high-cardinality column), and frequent small streaming or batch appends. Effects: slow listing, one task per tiny file, metadata overhead, and slow reads.

Fixes:

- Before writing, reduce partitions: `coalesce(n)`, or `repartition("partition_col")` to get about one file per folder.
- Limit output file size with `spark.sql.files.maxRecordsPerFile`.
- Compact with a periodic job (read and rewrite), or use **Delta OPTIMIZE** and auto compaction.
- Use fewer, coarser partition columns.
- On read, Spark packs small files into partitions by `spark.sql.files.maxPartitionBytes` (128 MB) and `spark.sql.files.openCostInBytes` (4 MB), which reduces but does not remove the cost.

Target output files of about 128 MB to 1 GB.

Q9. Explain bucketing and when it helps.

Bucketing pre-shuffles and pre-sorts data by a key into a fixed number of files when you write a **table** (`saveAsTable`), so later joins or aggregations on that key can **avoid the shuffle** when both sides are bucketed on the same key with the same bucket count.

```
(df.write.bucketBy(64, "customer_id").sortBy("customer_id")
   .mode("overwrite").saveAsTable("sales.orders_b"))
```

Trade-offs: the bucket count is fixed and hard to change, it only works for tables in a metastore (not plain paths), both sides must be compatible, and many engines now prefer Delta clustering features instead. It pays off for big tables joined repeatedly on the same key.

Q10. How does dynamic partition pruning work?

When a large partitioned fact table is joined with a small dimension table that has a selective filter, Spark cannot know at planning time which fact partitions are needed. With **dynamic partition pruning** (Spark 3.0+, `spark.sql.optimizer.dynamicPartitionPruning.enabled`, default true), Spark runs the filtered dimension side first (often as a broadcast), collects the matching join-key values and uses them at runtime to skip fact partitions that cannot match.

Example: `fact.join(dim.filter("region = 'EU'"), "dim_id")` with `fact` partitioned by `dim_id` reads only the partitions for EU dimension values. It requires the fact table to be partitioned on the join key and a join (usually a broadcast) with a selective filter.

Q11. How do you tune Spark file reads (maxPartitionBytes, schema merging, partition discovery)?

- `spark.sql.files.maxPartitionBytes` (128 MB) controls the input split size. Lower it for more parallelism on heavy per-row work, raise it for very light work.
- **Schema merging** (`mergeSchema=true` for Parquet) reads the footer of every file to unify schemas, which is expensive; use it only when schemas truly differ, or maintain a single table schema.
- **Partition discovery** lists the directory tree; very deep or wide partition trees make this slow, so prefer a table format (Delta) with a transaction log or a metastore.
- Prune columns and push filters; avoid `select *` on wide tables.
- Prefer explicit schemas for text formats to avoid the extra inference scan.

Section C: Advanced Streaming and Pipelines

Q12. How do you achieve exactly-once semantics in Structured Streaming with PySpark?

You need a **replayable source** (Kafka offsets or file tracking), **checkpointing** of offsets and state, and an **idempotent or transactional sink**. A Delta sink is transactional, so a retried micro-batch is not duplicated.

For custom logic use `foreachBatch`, but remember the batch may be re-executed after a failure, so make the write idempotent: MERGE on a business key, or write with a deterministic `batch_id` and skip batches already applied.

```
def upsert(batch_df, batch_id):
    (DeltaTable.forName(spark, "silver.orders").alias("t")
       .merge(batch_df.alias("s"), "t.id = s.id")
       .whenMatchedUpdateAll().whenNotMatchedInsertAll().execute())

stream.writeStream.foreachBatch(upsert) \
      .option("checkpointLocation", "/chk/orders").start()
```

Do not share a checkpoint between queries, and be careful that changing stateful logic usually needs a new checkpoint.

Q13. Explain stateful streaming: watermarks, deduplication and stream-stream joins.

Stateful operators (windowed aggregations, `dropDuplicates`, stream-stream joins, `applyInPandasWithState` and `flatMapGroupsWithState`) store state across micro-batches in a **state store** that is checkpointed.

- **Watermark** (`withWatermark("event_time", "10 minutes")`) tells Spark how late data may be; state older than the max event time minus the delay is dropped, and later events for closed windows are discarded.
- **Deduplication**: `dropDuplicates(["id", "event_time"])` with a watermark keeps bounded state (or `dropDuplicatesWithinWatermark`).
- **Stream-stream joins**: buffer both sides; add watermarks on both and a time-range condition, otherwise state grows without limit. Outer joins emit unmatched rows only after the watermark passes.

For large state use the **RocksDB state store** provider (state on local disk instead of the JVM heap) and monitor state rows and memory per operator in `lastProgress`.

Q14. How do you build incremental pipelines and idempotent loads in PySpark?

- **Incremental sources**: Structured Streaming with `availableNow=True` and a checkpoint, a high-watermark column (`updated_at > last_run`), or Change Data Feed on Delta. Store the watermark or checkpoint reliably.
- **Idempotent writes**: MERGE by business key, or overwrite exactly the affected partition (`replaceWhere` on Delta, `partitionOverwriteMode=dynamic` on file tables) so reruns give the same result.
- **Deterministic logic**: avoid `current_timestamp()` or random values inside keys; pass the run date as a parameter.
- **Late data**: keep Bronze immutable, reprocess affected partitions, or use watermarks in streams.
- **Backfills**: parameterised by date range, bounded in size, and validated with reconciliation counts.

Q15. How do you implement SCD Type 2 in PySpark?

Close the current row when a tracked attribute changes and insert a new current row. With Delta's MERGE:

```
from delta.tables import DeltaTable
dim = DeltaTable.forName(spark, "gold.dim_customer")

changed = (updates.alias("s")
   .join(dim.toDF().alias("t"),
         (F.col("s.id") == F.col("t.id")) & F.col("t.is_current"))
   .filter("s.address <> t.address")
   .select("s.*"))

staged = (changed.withColumn("merge_key", F.lit(None).cast("int"))
   .unionByName(updates.withColumn("merge_key", F.col("id"))))

(dim.alias("t")
   .merge(staged.alias("s"), "t.id = s.merge_key AND t.is_current")
   .whenMatchedUpdate(condition="s.address <> t.address",
        set={"is_current": "false", "end_date": "current_date()"})
   .whenNotMatchedInsert(values={"id": "s.id", "address": "s.address",
        "is_current": "true", "start_date": "current_date()",
        "end_date": "null"})
   .execute())
```

The trick: changed rows appear twice in the staged set, once with a null merge key (so they insert the new version) and once with the real key (so they close the old version). Deduplicate the source first and order by an event timestamp if there are several changes per key.

Section D: Debugging and Production Issues

Q16. What are common PySpark errors and how do you fix them?

- **AnalysisException: cannot resolve column / ambiguous reference**: misspelt column, or duplicate names after a join. Alias the DataFrames or join on the column name.
- **Py4JJavaError with OutOfMemoryError**: see the memory answers; check driver versus executor, skew and `collect`.
- **Container killed for exceeding memory limits**: raise `memoryOverhead` and Python memory settings.
- **Python worker exited unexpectedly / crashed**: usually Python-side memory or a library crash inside a UDF.
- **Broadcast timeout / "Cannot broadcast the table that is larger than"**: the table is too big or slow to compute; remove the hint, raise the timeout or filter first.
- **FileNotFound / "underlying files have been updated"**: files changed during a read; rerun or refresh the table.
- **Schema mismatch on union or write**: use `unionByName` and explicit casts.
- **Too many open files / fetch failures**: too many shuffle partitions or lost executors; adjust partitions and check node health.

Q17. Scenario: you need to join two very large tables (1 TB each) and the job fails with OOM or runs forever. What do you do?

- **Reduce before joining**: select only needed columns, filter early, and aggregate one side if possible.
- **Check skew**: look at the key distribution; remove or isolate hot keys (nulls and defaults), enable AQE skew join, or salt.
- **Size partitions properly**: raise shuffle partitions so each is about 100 to 200 MB and avoid spill.
- **Prefer sort-merge** for two big sides, and make sure statistics and partition filters allow pruning.
- **Avoid repeated joins**: bucket or cluster both tables on the join key, or pre-join once and persist.
- **Right-size memory** and overhead; use more, smaller executors if partitions are fine but GC is heavy.
- **Verify** in the Spark UI: task time distribution, spill, shuffle sizes.

Q18. Scenario: design a daily PySpark pipeline for about 1 TB of new data. What are your decisions?

- **Ingest**: land raw data as is (immutable Bronze), use incremental ingestion (file tracking or Kafka offsets) instead of rescanning history.
- **Format and layout**: Parquet or Delta; partition by date only (coarse), cluster or Z-order by the main filter columns; target files of 128 MB to 1 GB.
- **Processing**: explicit schemas, early filters and column selection, broadcast joins for dimensions, AQE on, built-in functions rather than UDFs.
- **Cluster**: size for about 100 to 200 MB per partition, autoscale, use spot nodes for workers where acceptable, enough memory overhead for PySpark.
- **Reliability**: idempotent writes (MERGE or partition overwrite), retries, data-quality gates, quarantine for bad rows, alerts on duration and counts.
- **Operations**: parameterised runs for backfill, logging of row counts, cost tagging, and a documented runbook.

Q19. Scenario: deduplicate billions of events with late-arriving updates. What is your approach?

- Define the business key and the ordering column (event time or version).
- For batch: window `row_number` over key ordered by version descending and keep 1, but shuffling billions of rows is costly, so first **limit the data**: only reprocess the partitions that could contain updated keys.
- Maintain a **Delta target and MERGE** only the new batch (deduplicated within the batch first), so history is not recomputed.
- For streaming: `dropDuplicates` with a watermark bounds state, or `foreachBatch` MERGE for late updates.
- Handle ties deterministically (add a tiebreaker such as ingestion time), and keep the raw events so you can replay.
- Validate with counts of distinct keys before and after.

Q20. Scenario: how do you handle schema changes in incoming files?

- Land data in Bronze with a tolerant reader (`PERMISSIVE` mode, the corrupt-record column, or Auto Loader's rescued-data column) so no data is lost.
- Use `unionByName(allowMissingColumns=True)` and `mergeSchema` (or table schema evolution in Delta) deliberately, not by default everywhere.
- In Silver, enforce an explicit schema and cast columns; send rows that fail casts to a quarantine table.
- Monitor schema drift (compare incoming columns to the expected list) and alert.
- Agree a data contract with the producer, and version breaking changes.

Principle: be flexible at the edge, strict in the core layers.

Section E: Coding Challenges (Advanced)

Q21. Coding: sessionization (new session after 30 minutes of inactivity).

```
w = Window.partitionBy("user").orderBy("ts")

s = (events
  .withColumn("prev_ts", F.lag("ts").over(w))
  .withColumn("new_session",
      F.when(F.col("prev_ts").isNull() |
             (F.col("ts").cast("long")
              - F.col("prev_ts").cast("long") > 1800), 1).otherwise(0))
  .withColumn("session_id", F.sum("new_session").over(w)))
```

The cumulative sum of the "new session" flags gives a session number within each user. Combine with `user` for a global session key, and aggregate per session for duration and event count.

Q22. Coding: gaps and islands (longest streak of consecutive login days per user).

```
days = logins.select("user", F.to_date("ts").alias("d")).distinct()
w = Window.partitionBy("user").orderBy("d")

g = (days
  .withColumn("rn", F.row_number().over(w))
  .withColumn("grp", F.expr("date_sub(d, rn)")))

streaks = (g.groupBy("user", "grp")
  .agg(F.count("*").alias("len"), F.min("d").alias("start"),
       F.max("d").alias("end")))
best = streaks.withColumn("r",
    F.row_number().over(Window.partitionBy("user").orderBy(F.desc("len")))
  ).filter("r = 1")
```

Consecutive dates minus an increasing row number give a constant value, so that constant identifies each island of consecutive days.

Q23. Coding: top 3 products per category by revenue, safely on skewed data.

```
rev = (sales.groupBy("category", "product")
            .agg(F.sum("amount").alias("revenue")))

w = Window.partitionBy("category").orderBy(F.desc("revenue"))
top3 = rev.withColumn("rn", F.row_number().over(w)).filter("rn <= 3")
```

Aggregate first so the window only sees one row per (category, product), which is much smaller than the raw sales data and avoids shuffling raw rows by a skewed `category`. If one category is still huge, the window ranks only its aggregated product rows, which is cheap.

Q24. Coding: broadcast a lookup map instead of a join, and when is that appropriate?

```
from itertools import chain
mapping = {"IN": "India", "US": "United States", "UK": "United Kingdom"}
m = F.create_map([F.lit(x) for x in chain(*mapping.items())])

df = df.withColumn("country_name", m[F.col("code")])
```

For a tiny, static lookup (tens to a few thousand entries), a literal map expression is evaluated inside the JVM with no join and no shuffle. For larger or changing lookups, use a normal broadcast join on a DataFrame instead.

Section F: Engineering Judgement and Behavioural Questions

Q25. When would you use the pandas API on Spark (pyspark.pandas) versus pandas versus Spark DataFrames?

- **pandas**: data fits on one machine; fastest to develop; ideal for small data and final presentation steps.
- **pandas API on Spark** (`import pyspark.pandas as ps`): pandas-style syntax that runs on Spark; useful for migrating pandas code to big data with minimal change.
- **Spark DataFrames and SQL**: the most control and best optimisation; the right choice for production pipelines.

Pitfalls of the pandas API on Spark: operations that assume a global row order or an index can be expensive (the default index can force a global sort or sequence), some pandas behaviours differ, and calling `.to_pandas()` collects everything on the driver. For production, use native Spark DataFrames and set an explicit index only if needed.

Q26. Compare Spark with Hadoop MapReduce and Flink. What are the deploy modes and cluster managers?

Spark vs MapReduce: Spark keeps intermediate data in memory and uses a DAG, so it is usually much faster, especially for iterative and interactive work, and has one API for batch, SQL, streaming and ML. MapReduce writes every stage to disk and has only map and reduce steps.

Spark vs Flink: Flink is a true record-at-a-time streaming engine with very low latency and rich state handling; Spark Structured Streaming is micro-batch based (with an experimental continuous mode), with a unified batch and stream API and a huge ecosystem. Choose by latency need and team skills.

Cluster managers: Standalone, YARN, Kubernetes (Mesos is deprecated). **Deploy modes**: client (driver on the submitting machine) and cluster (driver inside the cluster; preferred for production and survives the client disconnecting).

Q27. Which newer Spark features should a senior engineer know?

- **AQE** and **dynamic partition pruning** (Spark 3.x), which reduce manual tuning.
- **Pandas API on Spark**, and improved pandas UDFs with type hints.
- **Spark Connect** (3.4+): a client-server architecture so thin clients can run Spark remotely.
- **ANSI SQL mode**: stricter semantics (errors on overflow and invalid casts instead of silent nulls); the default in Spark 4.0.
- **VARIANT type** for semi-structured data and the **Python data source API** (Spark 4.0).
- **Structured Streaming** improvements: state store enhancements, `availableNow` trigger and arbitrary stateful processing in Python.

Versions evolve quickly, so say which version you used and mention that you verify the release notes before adopting a feature.

Q28. Tell me about a PySpark job you optimised. How should you structure the answer?

Use **STAR** with real numbers.

- **Situation**: "A daily customer-churn pipeline took 4 hours and kept missing its SLA."
- **Task**: "Reduce runtime and make it reliable."
- **Action**: "I opened the Spark UI and found one stage with a long tail: one key held 40 percent of the rows. I filtered null keys, broadcast the small dimension table, enabled AQE skew handling, replaced a Python UDF with built-in functions, wrote Parquet with a sensible partition column instead of thousands of small files, and cached one DataFrame reused three times."
- **Result**: "Runtime dropped from 4 hours to 35 minutes, cost fell by about 60 percent, and failures stopped."

Interviewers look for a method (measure, diagnose, fix, verify), not just a list of tricks. Prepare two or three real stories in advance.

Q29. What are the most common mistakes in production PySpark code?

- Using `collect()` or `toPandas()` on large data.
- Python UDFs where built-in functions or pandas UDFs would do.
- Ignoring skew and leaving the default 200 shuffle partitions for terabyte data.
- Over-partitioning output and creating millions of small files.
- Reading CSV or JSON with schema inference in production.
- Joins on duplicate column names without aliases, or joins that silently explode because the key is not unique.
- Caching everything and never unpersisting.
- Non-idempotent writes that duplicate data on retry.
- Hard-coded paths, dates and credentials instead of parameters and secrets.
- No tests, no row-count reconciliation, no alerts.

Appendix A: Rapid-Fire Reference (Defaults and Numbers)

Numbers interviewers like to ask. Defaults can change between runtime versions, so treat these as a revision aid and confirm in the docs.

Topic	Value
spark.sql.shuffle.partitions	200 (default)
Auto broadcast join threshold	10 MB (spark.sql.autoBroadcastJoinThreshold)
Broadcast timeout	300 seconds (spark.sql.broadcastTimeout)
Max bytes per input partition	128 MB (spark.sql.files.maxPartitionBytes)
File open cost	4 MB (spark.sql.files.openCostInBytes)
AQE	Enabled by default since Spark 3.2
AQE skew rule (default)	Partition over 5 times the median and over 256 MB
Unified memory fraction	0.6 of heap after 300 MB reserved (spark.memory.fraction)
Storage fraction	0.5 of unified memory (spark.memory.storageFraction)
Executor memory overhead	Max of 384 MB or about 10 percent of executor memory
DataFrame cache() level	MEMORY_AND_DISK
RDD cache() level	MEMORY_ONLY
Task retries	4 (spark.task.maxFailures)
Speculation	Off by default (spark.speculation)
Locality wait	3 seconds (spark.locality.wait)
Arrow batch size	10000 records (spark.sql.execution.arrow.maxRecordsPerBatch)
Default write mode	errorifexists
Default JSON/CSV read mode	PERMISSIVE
Streaming output modes	append, update, complete
Join types	inner, left, right, full, left_semi, left_anti, cross

Appendix B: Final Interview Checklist

- Be able to **explain the execution model**: driver, executors, DAG, jobs, stages, tasks and shuffles.
- Know the **DataFrame API cold**: select, filter, withColumn, groupBy/agg, joins, window functions, explode and null handling.
- Know **why** each optimisation works: broadcast joins, AQE, pruning and pushdown, partition sizing, caching.
- Be able to **read the Spark UI and explain()** and match symptoms (skew, spill, small files, shuffle) to fixes.
- Practise **writing code live**: dedup with window functions, top N per group, running totals, word count, left anti join, sessionization.
- Prepare **three real stories**: one performance win, one production failure you fixed, one design decision with trade-offs.
- For scenario questions, **clarify data volume, latency and SLA first**, then design, then discuss cost, testing and monitoring.
- Be honest about what you have and have not used in production.